



UNIVERSIDAD DE LOS LAGOS

9

Programación Orientada a Objetos

Docente: Patricio Araneda García

Correo: patricio.aranedagarcia@ulagos.cl



PATRICIO ARANEDA GARCÍA

Magíster en Informática Médica

Universidad de Chile

DATACLINICA Spa



patricioaraneda.cl



dataclinica.cl



/in/patricioaraneda



@paranedagarcia

Profesor de Ciencias Naturales y Biología

Pontificia Universidad Católica de Chile

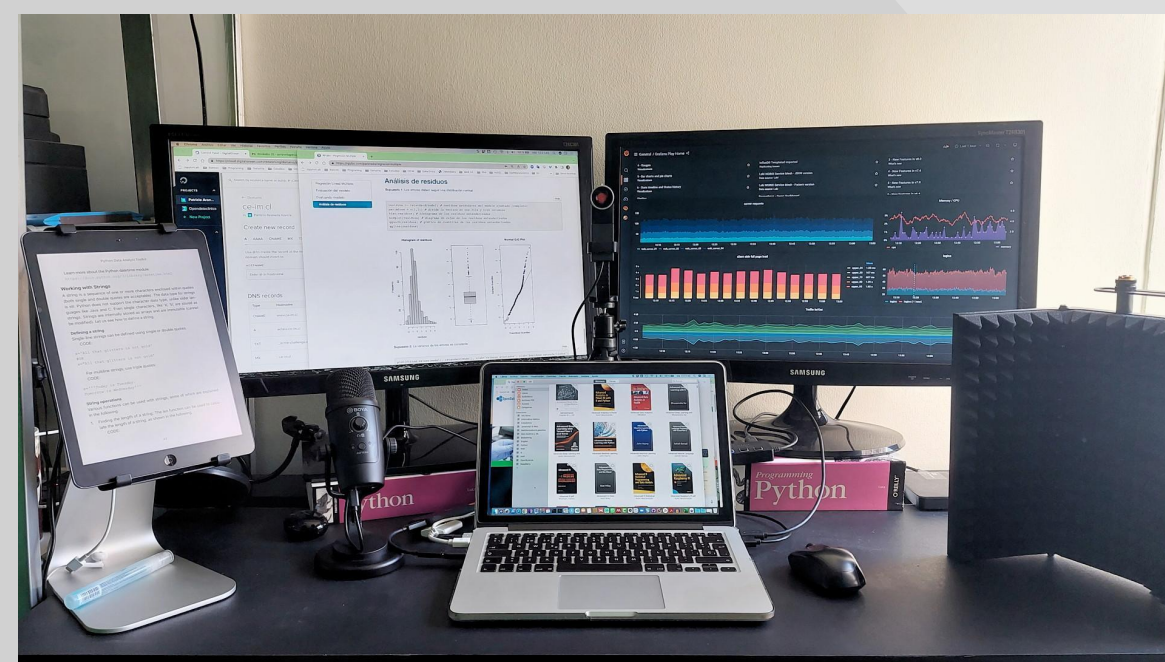
Ingeniero en Computación e Informática

Instituto Profesional Virginio Gómez

Magíster en Informática Médica

Universidad de Chile

- Salud Pública y Redes Asistenciales
- Inteligencia Artificial
- Modelamiento Productivo y Machine Learning



<https://github.com/paranedagarcia>



<https://rpubs.com/paraneda>

DOCENCIA UNIVERSITARIA

- Análisis de Datos
- Bases de Datos
- Fundamentos de Sistemas Inteligentes
- Sistemas y Tecnologías de Información
- Ciencia de Datos y Machine Learning
- Inteligencia Artificial
- Bioestadística



UNIVERSIDAD DE LOS LAGOS

CONTENIDOS DEL CURSO

Manual de Ingeniería Informática

Desarrolla sobre las Tecnologías de la Información

Manual



Fundamentos

Conozca los conceptos básicos de la Ingeniería de la Información y aplique para la resolución de problemas en cualquier ámbito.



Programación

Aprenda las bases de la programación, algoritmos y estructuras de datos, incluyendo la sintaxis, estructuras de control y conceptos fundamentales.



Gobernanza

Aprenda las mejores prácticas para la gobernanza de los datos e información en el ámbito organizacional (en construcción...).

Otros cursos de la serie	Conecta	Más	Licencia
Ingeniería Informática	Website	GitHub	CC BY-NC-SA 4.0 - Solo para uso docente y académico. No olvides citar la fuente y respetar la licencia. Deja una estrella en GitHub si te gustó el curso. ¡Gracias!
Programación en Python	LinkedIn	ORCID	
Bioestadística	Whatsapp	Correo	
Ciencia de Datos			
Análisis con SQL			

<https://patricioaraneda.cl/informatica>

- Programación

Programando con Python

Desde Ciencia de Datos hasta Machine Learning

Curso Python



Programación

Este curso fue diseñado desde cero para ser fácilmente accesible y comprensible para todos. No requiere conocimientos previos de estadística o programación.



Data Science

Se enfoca en la aplicación de la estadística en la investigación médica, y cubre los conceptos y técnicas estadísticas más relevantes para el análisis de datos en este campo.



Desarrollo Continuo

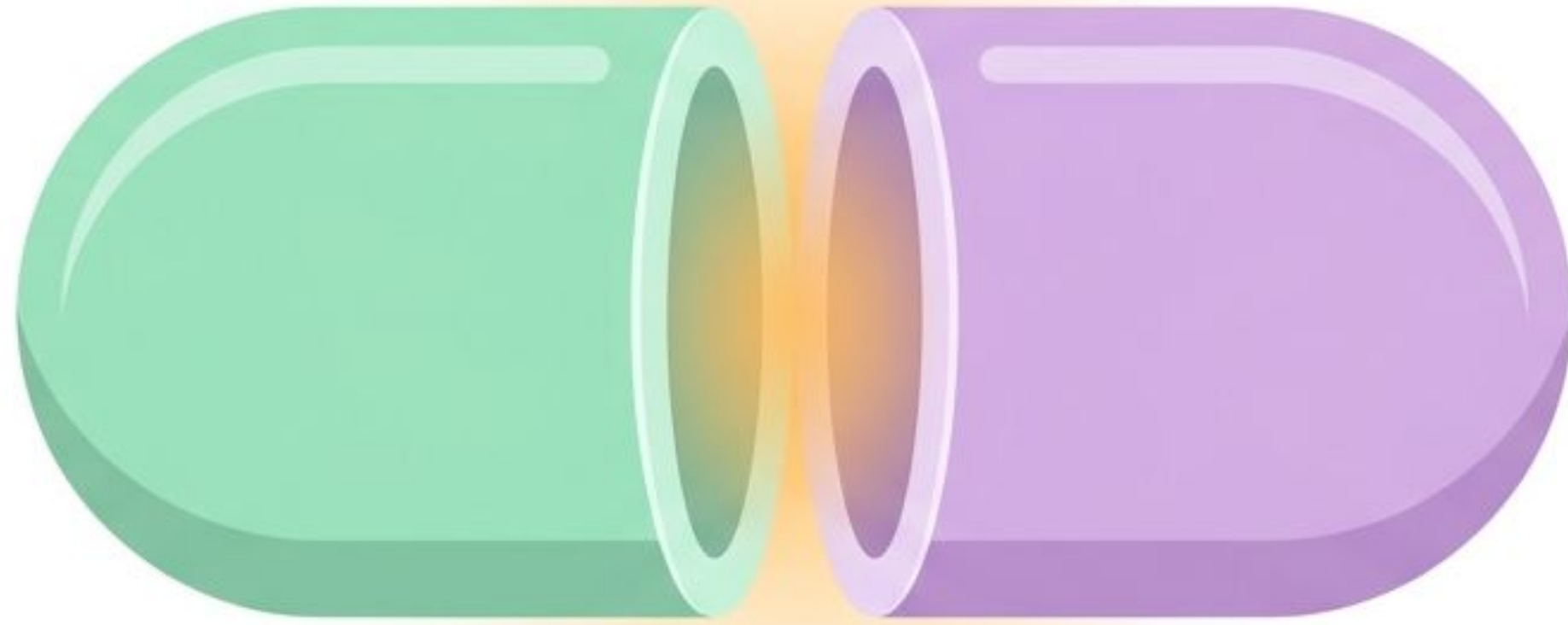
Esta en constante desarrollo, y se irán agregando nuevos contenidos y recursos a medida que se avance en el curso. Se anima a los estudiantes a participar activamente en el curso, compartiendo sus experiencias y sugerencias.

Otros cursos de la serie	Conecta	Más	Licencia
Ingeniería Informática	Website	GitHub	CC BY-NC-SA 4.0 - Solo para uso docente y académico. No olvides citar la fuente y respetar la licencia. Deja una estrella en GitHub si te gustó el curso. ¡Gracias!
Programación en Python	LinkedIn	ORCID	
Bioestadística	Whatsapp	Correo	
Ciencia de Datos			
Análisis con SQL			

<https://patricioaraneda.cl/python/docs/poo/>

The background of the slide is a complex digital illustration. It features several server racks, some with glowing green lights. A central hub of glowing orange and yellow light is connected by numerous thin, translucent blue lines to several green, spherical nodes. These nodes are further connected to the server racks. The background is filled with abstract digital patterns, including glowing blue and orange lines, and faint, stylized text that appears to be code or data. The overall color scheme is dominated by blue, green, and orange, creating a high-tech, futuristic feel.

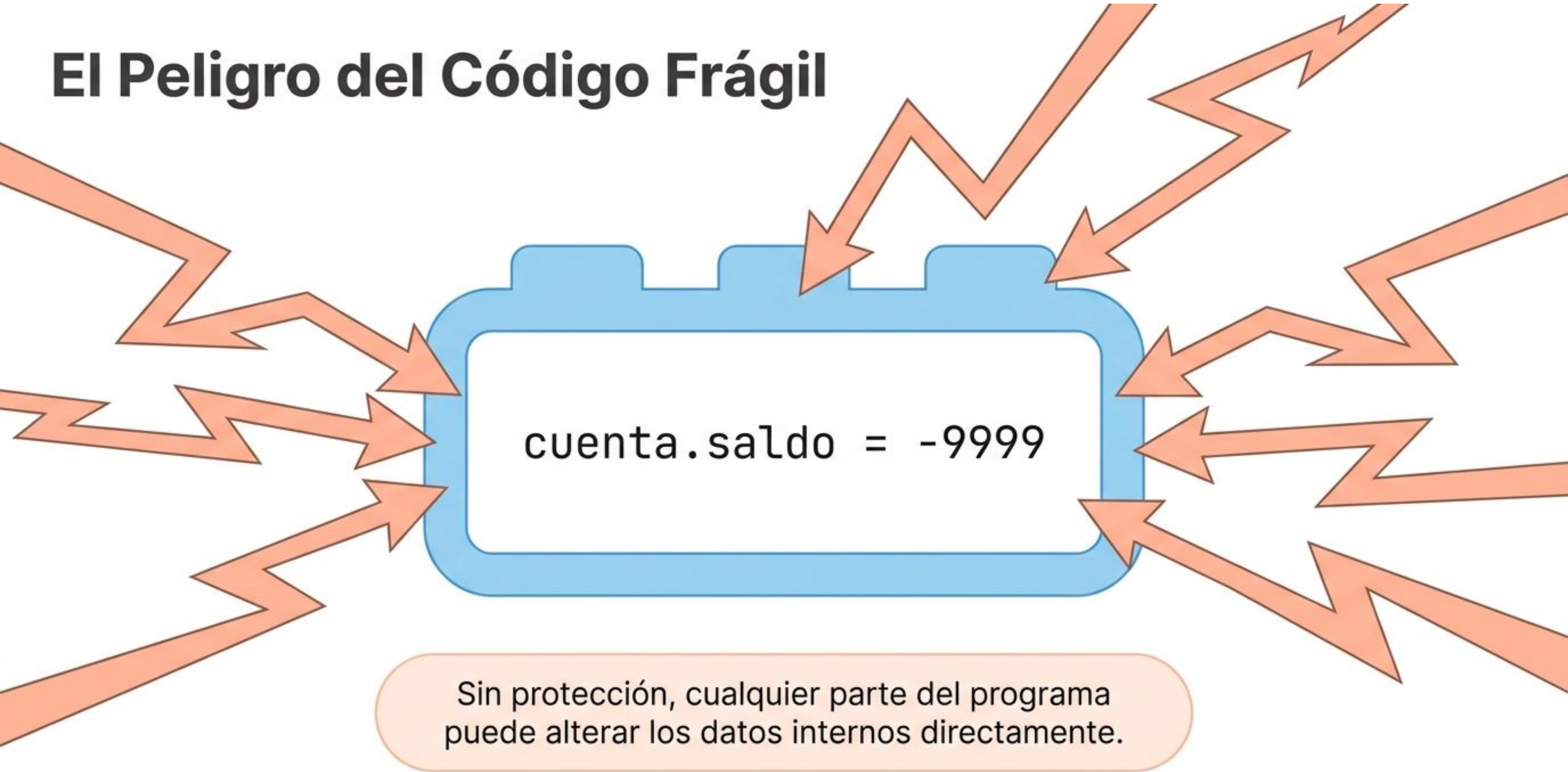
Encapsulamiento



Encapsulamiento en Python

Domina la privacidad de los datos en la
Programación Orientada a Objetos.

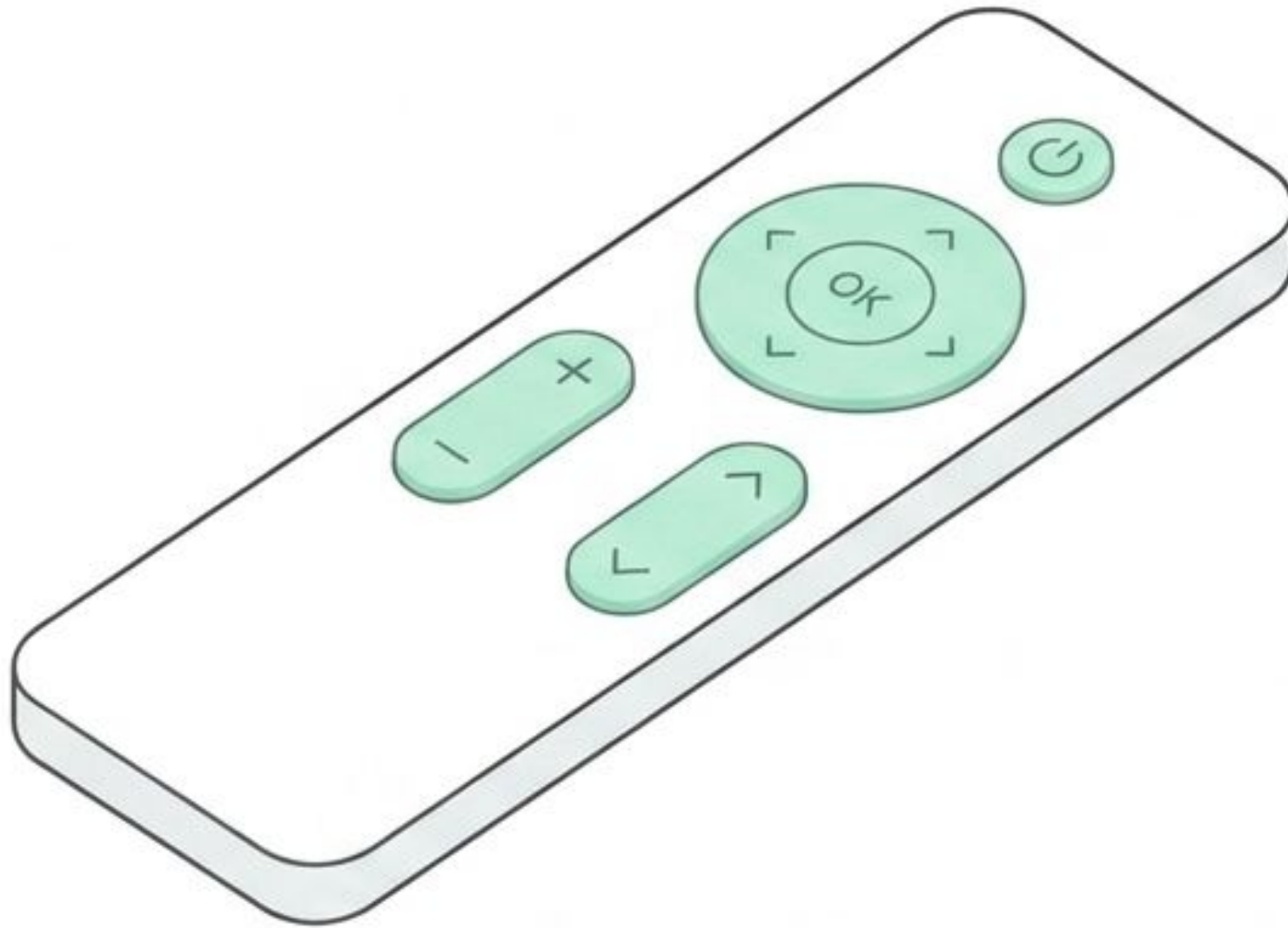
El Peligro del Código Frágil



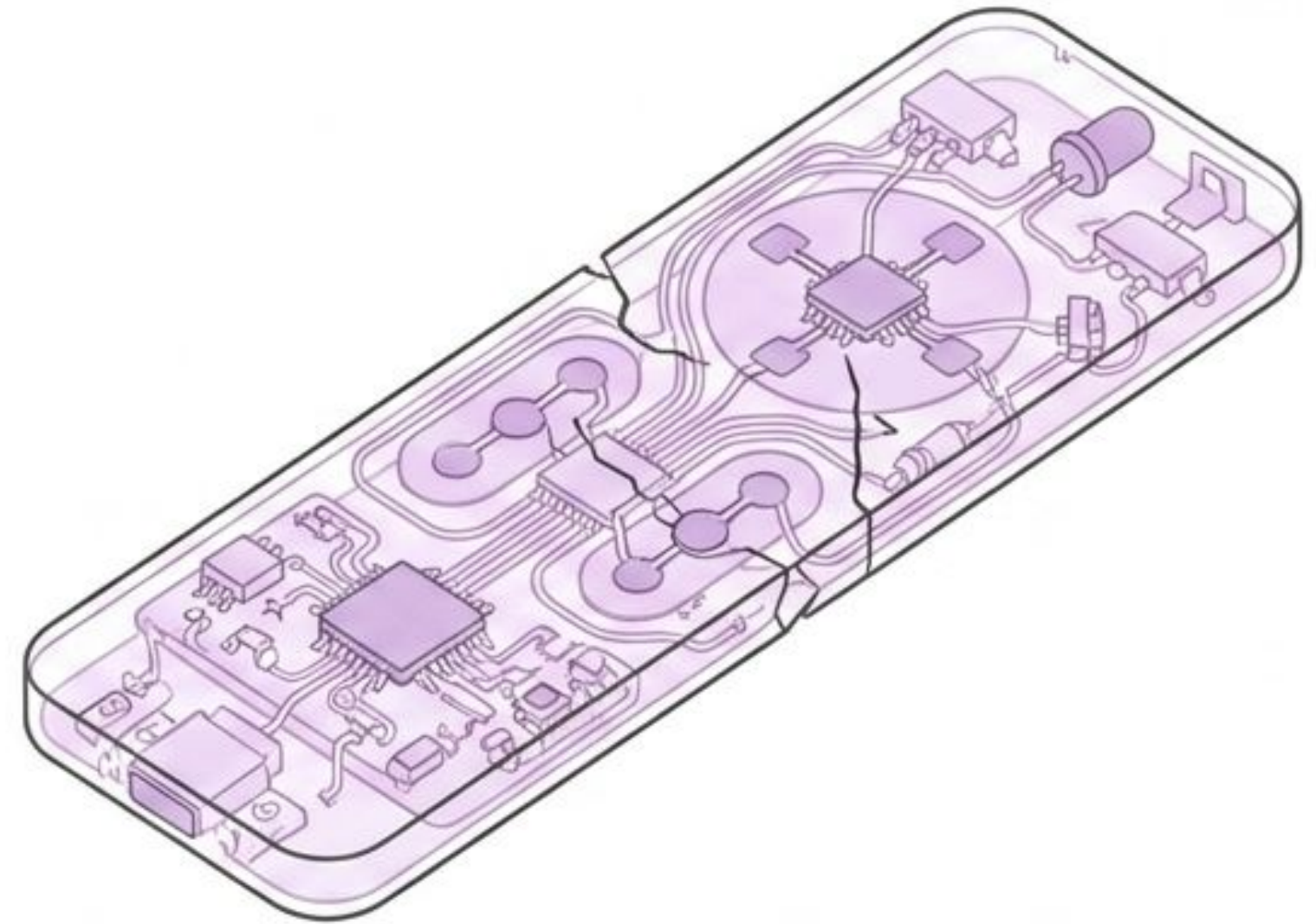
Esto crea estados inválidos o imposibles (ej. saldos negativos, edades irreales), rompiendo la lógica del sistema.

La Metáfora del Control Remoto

Interfaz Pública

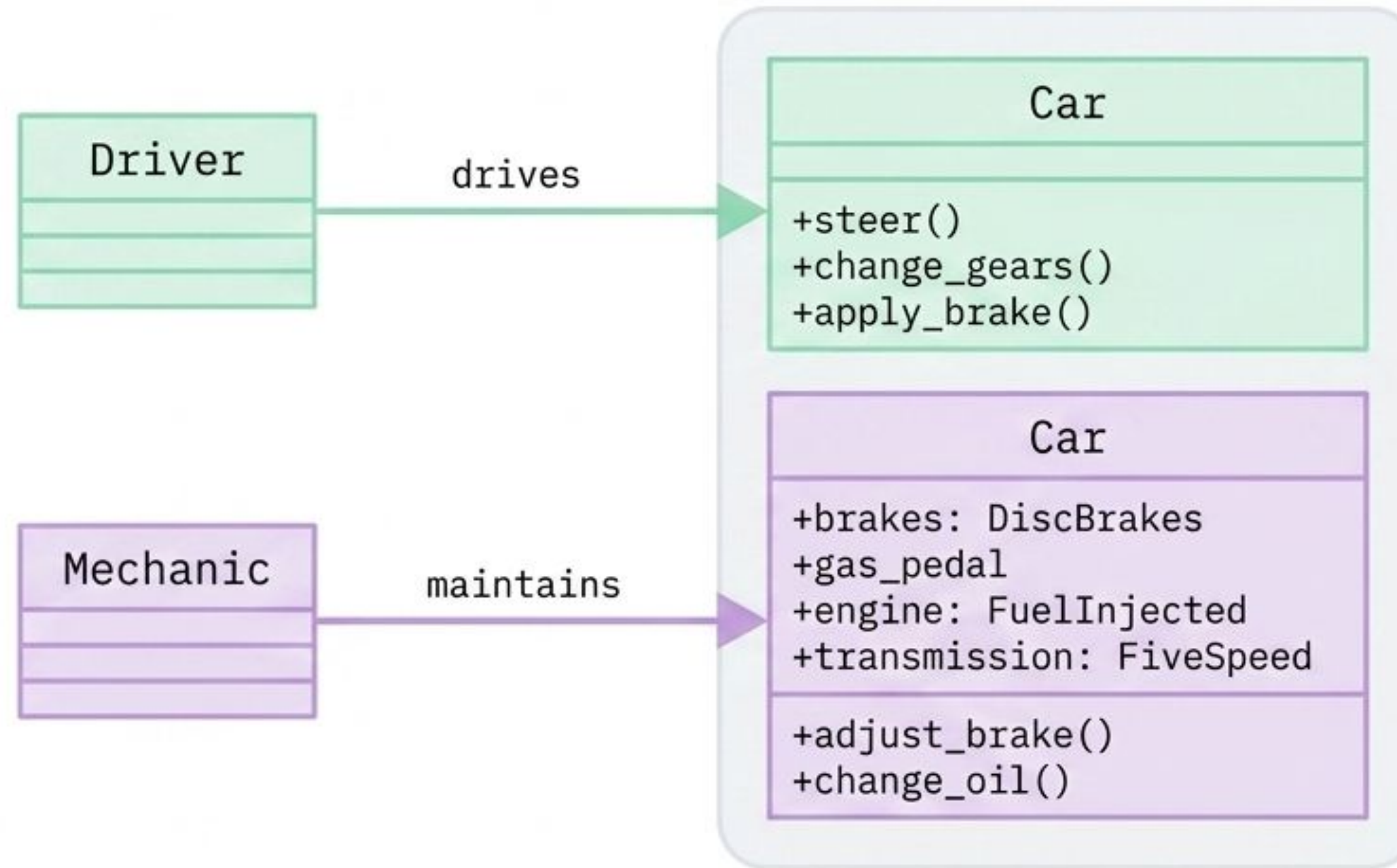


Lógica Oculta



Usas los botones (métodos) para cambiar de canal. No necesitas entender los circuitos internos (atributos y lógica) para que el televisor funcione.
El fabricante oculta la complejidad para tu comodidad.

¿Qué es el Encapsulamiento?



Es el principio de agrupar los datos y las acciones en un solo objeto, exponiendo solo lo necesario.

Abstracción:
El conductor interactúa con la interfaz pública. Los detalles del motor quedan ocultos (encapsulados).

TODOS SOMOS ADULTOS RESPONSABLES

Java/C++



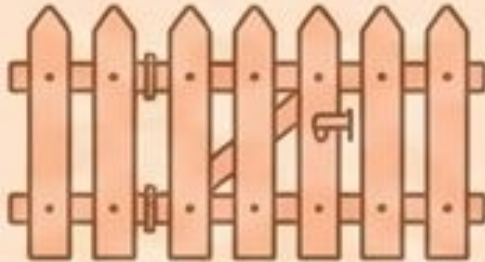
Python



A diferencia de otros lenguajes, Python no impone leyes estrictas con palabras clave para ocultar datos. Confía en la responsabilidad del programador mediante convenciones de nombres.

Los Tres Niveles de Acceso en Python

Una convención visual para comunicar la intención de diseño

Público	nombre		Acceso Total
Protegido	<code>_nombre</code>		Acuerdo de caballeros (Uso interno)
Privado	<code>__nombre</code>		Oculto (Name Mangling)

Convenciones en Acción

```
self.edad = 20
```

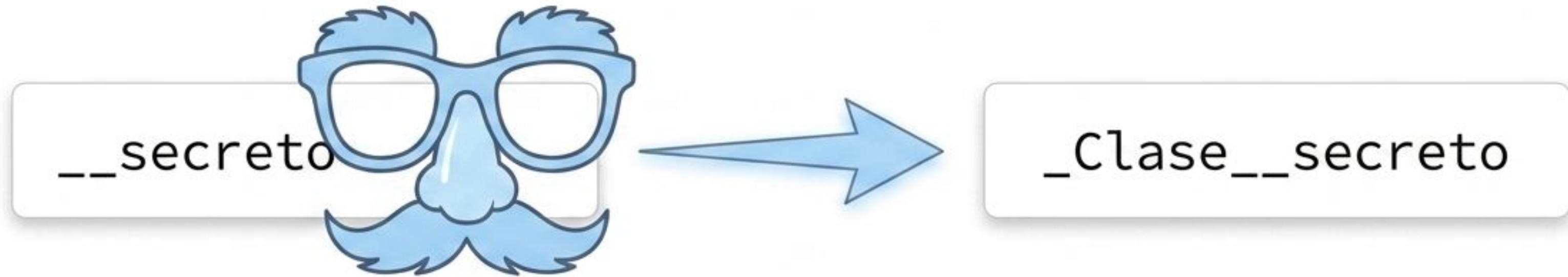
Público: Se lee y modifica libremente desde cualquier parte.

```
self._sueldo = 1000
```



Protegido (_): El guión bajo simple es una señal de advertencia. Técnicamente puedes acceder, pero si rompes el código, es tu culpa.

El Doble Guión (__)



El Name Mangling. Python cambia internamente el nombre de los atributos con doble guión bajo (__) añadiendo el nombre de la clase.

No es seguridad **criptográfica impenetrable**, es **prevención** para **evitar colisiones accidentales** de variables durante la **herencia**.

Olvidemos los Getters Clásicos

```
def get_edad(self):  
    return self._edad  
  
def set_edad(self, val):  
    self._edad = val
```

En otros lenguajes, creas métodos largos y repetitivos para leer o escribir cada variable. En Python, esto **ensucia el código**, **reduce la legibilidad** y se considera un **antipatrón**.

La Solución: @property



Permite acceder a los métodos como si fueran atributos simples.
 Obtienes la sintaxis **limpia** de una variable, pero **con el poder**,
 la **validación** y el **control oculto** de un **método**.

Código Elegante y Seguro

El Lector (Getter)



```
@property
def edad(self):
    return self._edad
```

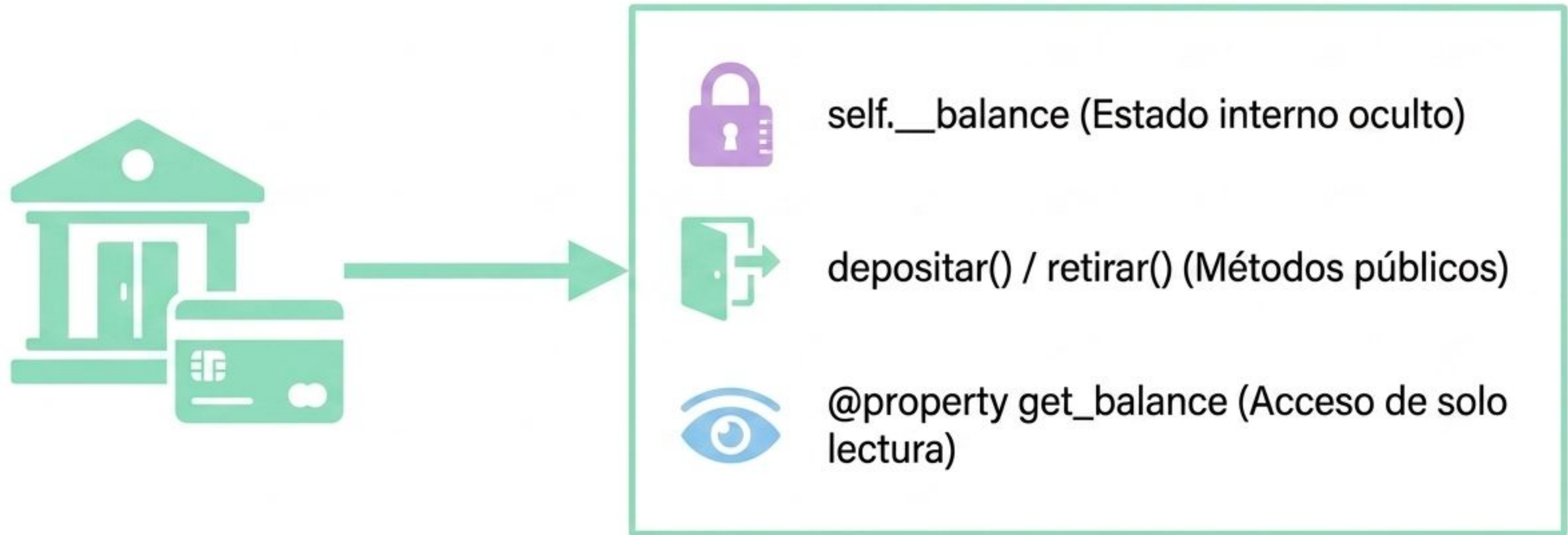
El Escritor con
validación (Setter)



```
@edad.setter
def edad(self, valor):
    if valor < 0:
        raise ValueError
    self._edad = valor
```

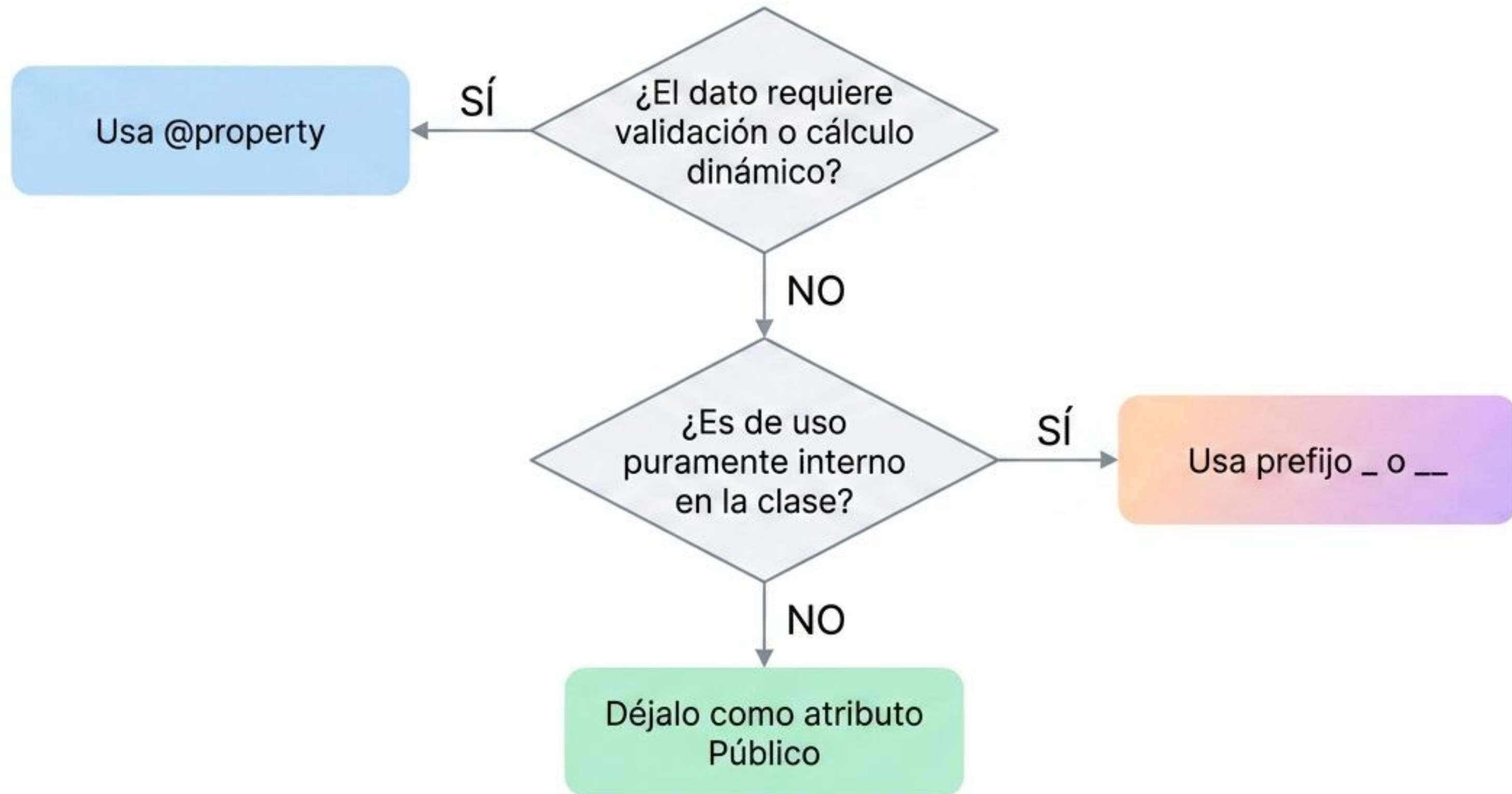
Si en el futuro necesitas validar un dato, agregas @property sin romper el código de quienes ya usan tu clase interactuando con obj.edad.

EJEMPLO: LA CUENTA BANCARIA



El saldo (`__balance`) está fuertemente encapsulado. El usuario solo puede interactuar mediante métodos formales (`depositar`). El resultado: es imposible tener un saldo negativo por un error de asignación directa.

¿CUANDO OCULTAR LOS DATOS?



Sigue este flujo de diseño moderno para mantener tus clases simples, legibles y Pythonicas.

EL KIT DEL BUEN PYTHONISTA

Resumen de las mejores prácticas para dominar la privacidad en la Programación Orientada a Objetos.



Confianza

Privacidad por convención, no por fuerza.



Legibilidad

Acceso directo por defecto, evita el código basura.



Evolución

Usa `@property` para añadir control sin romper interfaces.



Seguridad

Protege la lógica crítica (ej. saldos) con `_` o `__`.

DOMINA TUS OBJETOS



El encapsulamiento no se trata de esconder por esconder, sino de diseñar interfaces claras, seguras y fáciles de usar.